

AI-Assisted Software Development

Module 5: Best Practices and Tips

Mike Burton

Overview

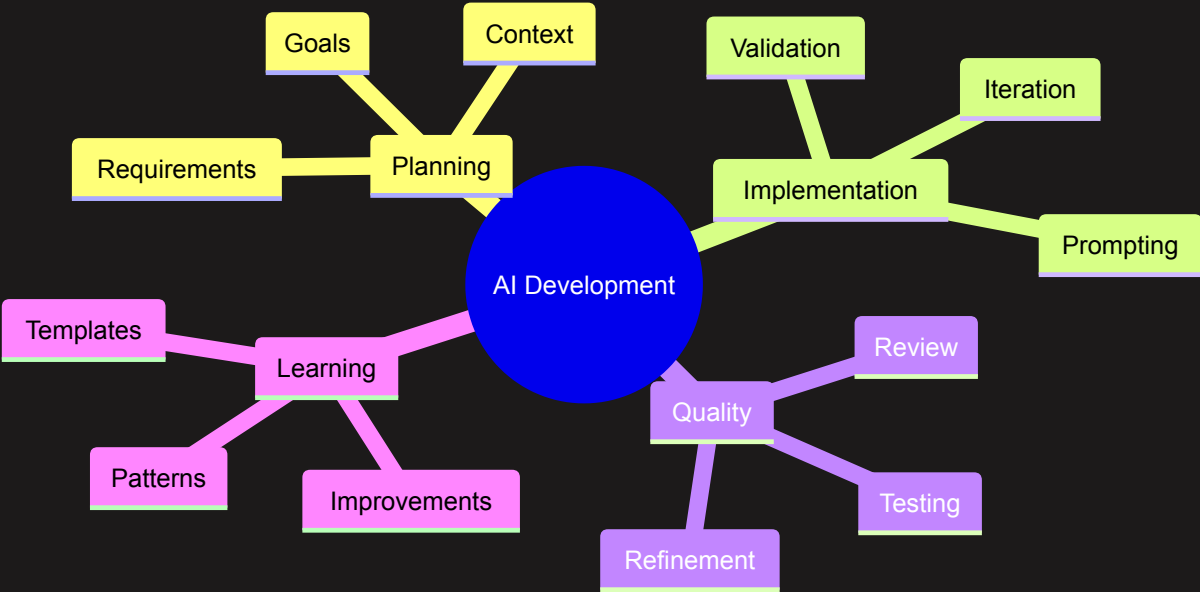
What we'll cover:

- Effective prompting
- Context management
- Workflow optimization
- Common pitfalls
- Real-world tips

Key outcomes:

- Better AI interactions
- Faster development
- Higher quality code
- Consistent results
- Practical workflows

The AI Development Lifecycle



Effective Prompting Strategies

Example prompts that work well:

1. Feature Implementation

```
"Create a user authentication system with:
```

```
Context:
```

- Next.js 13 app
- Prisma ORM
- PostgreSQL database

```
Requirements:
```

- Email/password login
- OAuth (Google, GitHub)
- Password reset flow
- Session management
- Rate limiting

```
Include:
```

- Database schema
- API endpoints
- Error handling
- Security measures"

2. Code Review

```
"Review this authentication implementation:  
[paste code]"
```

Context Management

Consider:

- Security best practices
- Error handling
- Edge cases
- Performance
- Scalability

1. Project Setup

Provide specific improvements with code examples."

```
/project
├── docs/
│   ├── requirements.md
│   ├── architecture.md
│   └── api-spec.md
├── src/
│   ├── features/
│   └── components/
└── tests/
```

2. Documentation

```
Feature: User Management
Status: In Development
Dependencies:
- Authentication
- Database
- Email Service
```

3. AI Context

When working on feature X:

1. Open related files
2. Reference documentation
3. Include test files
4. Specify constraints
5. Mention dependencies

3 / 16

4. Version Control

Commit messages that help AI:

- feat(auth): implement login flow
- fix(api): handle rate limiting
- refactor(db): optimize queries

Real-World Workflow Examples

1. Feature Development

Step 1: Planning

"Design an email notification system:

- Types of notifications
- Template system
- Queue management
- Error handling
- Rate limiting"

Step 2: Implementation

"Generate the notification service with:

- Queue processor
- Template engine
- Retry mechanism
- Logging system"

Step 3: Testing

"Create tests covering:

- Template rendering
- Queue processing
- Error scenarios
- Rate limiting"

Code Generation Best Practices

1. Clear Requirements

```
"Create a React form component that:
```

```
Input:
```

- Handles user registration
- Validates email/password
- Shows field errors
- Manages submit state

```
Output needed:
```

- TypeScript types
- Form validation
- Error handling
- Loading states
- Success feedback"

2. Iterative Refinement

```
"Improve this form component by:
```

1. Adding field validation
2. Implementing error messages
3. Adding loading states
4. Including success feedback
5. Handling network errors"

Testing and Validation

Example workflow:

1. Generate Tests

```
"Create unit tests for this payment processor:  
[paste code]"
```

```
Include tests for:
```

- Successful payment
- Invalid card
- Insufficient funds
- Network errors
- Timeout scenarios
- Duplicate charges"

2. Validate Implementation

```
"Review these tests for:
```

- Coverage gaps
- Edge cases
- Error scenarios
- Integration points
- Performance impacts"

Common Patterns and Templates

1. API Endpoint Template

```
"Create an API endpoint for [feature]:
```

```
Requirements:
```

- Authentication required
- Input validation
- Rate limiting
- Error handling
- Response format
- Cache strategy
- Logging

```
Include:
```

- Request/response types
- Validation rules
- Error cases
- Example usage"

2. Component Template

```
"Create a [component] with:
```

```
Props:
```

- Required/optional props
- Type definitions
- Default values

Error Prevention Strategies

- Features:
- Error states
 - Loading states
 - Empty states
 - Success states

1. Input Validation

- Includes:
- Styling
 - Accessibility
 - Documentation
- "Review this validation logic:
[paste code]"

Check for:

- Missing edge cases
- Type coercion issues
- Security implications
- Performance impact
- User experience"

8 / 16

2. Error Handling

"Improve error handling for:
[paste code]"

Consider:

- User feedback
- Recovery options
- Logging needs
- Security aspects
- Performance impact"

9 / 16

Performance Optimization

Real-world optimization prompts:

1. Database Queries

```
"Optimize these queries:  
[paste queries]"
```

```
Focus on:
```

- Index usage
- Join efficiency
- Query planning
- Cache strategy
- Connection pooling"

2. Frontend Performance

```
"Review this React component:  
[paste component]"
```

```
Optimize for:
```

- Render efficiency
- Memory usage
- Bundle size
- Network calls
- State management"

Security Best Practices

Example security prompts:

1. Authentication Review

```
"Review this auth implementation:  
[paste code]"
```

Check for:

- Token management
- Session security
- Password handling
- Rate limiting
- OWASP compliance"

2. Data Protection

```
"Audit this data handling:  
[paste code]"
```

Verify:

- Input sanitization
- Access control
- Data encryption
- Audit logging
- Privacy compliance"

Common Pitfalls to Avoid

1. Prompt Issues

- ✗ "Make it secure"
- ✓ "Implement security following OWASP:
 - Input validation
 - Authentication
 - Authorization
 - Data protection
 - Error handling"

2. Context Problems

- ✗ "Fix the bug"
- ✓ "Fix the authentication bug:
 - Current behavior
 - Expected behavior
 - Error messages
 - Related code
 - Test cases"

Tips for Success

1. Effective Communication

Clear prompt structure:

1. Context
2. Requirements
3. Constraints
4. Expected output
5. Examples

2. Quality Assurance

Validation checklist:

- Code review
- Test coverage
- Performance check
- Security audit
- Documentation

Building Your AI Workflow

1. Create Templates

Project templates:

- Requirements gathering
- Architecture design
- Component structure
- Testing strategy
- Documentation format

2. Maintain Knowledge Base

Document:

- Successful prompts
- Common patterns
- Best practices
- Lessons learned
- Team guidelines

Recap

- Use clear, specific prompts
- Maintain proper context
- Follow established patterns
- Validate AI output
- Document learnings
- Build reusable templates
- Focus on quality
- Prioritize security

Next Steps

- Create your prompt library
- Build workflow templates
- Establish team practices
- Set up quality checks
- Document processes
- Share learnings